RAG for Legal Documents

Notebook Walkthrough, Evaluation Fix & Free LLM Options

A complete reference based on the assignment notebook

1. Notebook Overview

This notebook builds an end-to-end Retrieval-Augmented Generation (RAG) system for legal documents — covering data loading, preprocessing, vector storage, LLM-based answering, and evaluation.

Pipeline at a Glance

Stage	Tool / Model	Output
Data Loading	LangChain DirectoryLoader + TextLoader	644 LangChain Documents
Preprocessing	regex, NLTK stopwords	Cleaned lowercase text
Chunking	RecursiveCharacterTextSplitter (1000/200)	55,348 chunks
Embedding	all-MiniLM-L6-v2 (HuggingFace)	384-dim vectors
Vector Store	Qdrant (cloud)	Indexed chunk store
LLM / Generation	Qwen2.5-3B-Instruct (local GPU)	Context-grounded answers
Evaluation	RAGAS (faithfulness, precision, recall)	Metric scores

2. Data Loading & Preprocessing

2.1 Dataset Structure

The corpus folder contains four sub-types of legal documents:

- **contractnli** — Non-disclosure and confidentiality agreements
- **cuad** — Contracts with annotated legal clauses
- **maud** — Merger and acquisition agreements
- **privacy_qa** — Privacy policy question-answering dataset

Benchmark JSON files (contractnli.json, cuad.json, maud.json) contain 6,695 question–answer pairs for evaluation.

2.2 Preprocessing Steps

Applied in sequence to every document:

Step	Function	Purpose
1	remove_emails()	Strip email addresses
2	remove_phone_numbers()	Strip phone/fax numbers
3	remove_special_characters()	Keep only alphanumeric + spaces

4	remove_extra_spaces()	Collapse whitespace
5	convert_to_lowercase()	Normalise case
6	remove_stop_words()	Remove NLTK English stop words

2.3 Exploratory Analysis Results

Metric	Value
Total documents	644
Average document length	8,505 words
Maximum document length	84,946 words
Minimum document length	142 words
Total chunks generated	55,348

Top 10 Most Frequent Words (after stop-word removal)

company · agreement · section · parent · party · date · time · merger · material · subsidiaries

TF-IDF Similarity Observations

- First 10 docs (same subfolder): avg similarity **0.109** — higher due to shared legal boilerplate
- 10 random docs (mixed subfolders): avg similarity **0.156**, max **0.519** — some near-duplicate clauses detected

3. Vector DB & RAG Chain

3.1 Embedding Model

- Model: **sentence-transformers/all-MiniLM-L6-v2**
- Dimension: 384 | Device: CUDA (GPU)
- Loaded via LangChain HuggingFaceEmbeddings

3.2 Qdrant Vector Store

- Collection name: *legal_docs*
- 55,348 chunks indexed with dense vectors
- Retriever: top-3 chunks per query (MMR or similarity)

3.3 LLM — Qwen2.5-3B-Instruct

- Loaded with *device_map='auto'* and float16 on GPU
- max_new_tokens = 128 (answer generation only)
- Wrapped as LangChain HuggingFacePipeline

3.4 RAG Prompt Template

```
You are an expert legal assistant. Answer the user's question
using only the provided context below.
If you do not know the answer, say you don't know.

Context:
{context}

Question:
{question}

Answer:
```

✓ The RAG chain: retriever | format_docs → prompt → LLM → StrOutputParser

4. Evaluation — Why All Metrics Were NaN

■ All RAGAS metrics returned NaN due to TimeoutError on every job (0–14).

Root Cause Analysis

RAGAS internally makes many structured LLM calls to compute each metric:

- **faithfulness** — breaks the answer into atomic statements, then asks the LLM whether each is supported by the context (NLI-style)
- **context_precision** — asks the LLM to judge relevance of each retrieved chunk

- **context_recall** — asks the LLM to map ground-truth sentences to context chunks

Three compounding problems caused the timeouts:

Problem	Detail
LLM too slow	Qwen2.5-3B generates ~10 tok/s for RAGAS's long internal prompts; RAGAS async executor times ou
max_new_tokens=128 too low	RAGAS needs 200–500 tokens for its reasoning steps; answers are cut off mid-judgment, causing pars
Echoed prompt in output	HuggingFacePipeline returns the full prompt + answer; RAGAS judges the entire string including the sy

5. The Fix — Split Generation vs. Judgment

The solution is to separate two roles that the original code conflated:

Role	Model	Why
Answer generation	Qwen2.5-3B (local RAG chain)	This is what is being evaluated — keep it unchanged
RAGAS judge LLM	Fast API model (see Section 6)	Handles RAGAS's structured prompts without timeout

5.1 Fixed evaluate_rag_pipeline() — Section 3.1.2

```
from ragas.metrics.collections import faithfulness, context_precision, context_recall
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper

# judge_llm = one of the options in Section 6
ragas_llm        = LangchainLLMWrapper(judge_llm)
ragas_embeddings = LangchainEmbeddingsWrapper(embedding_function)

def evaluate_rag_pipeline(questions, ground_truths):
    eval_data = {"question":[], "answer":[], "contexts":[], "ground_truth":[]}

    for i, (q, gt) in enumerate(zip(questions, ground_truths)):
        docs     = retriever.invoke(q)
        contexts = [d.page_content for d in docs]
        raw      = rag_chain.invoke(q)
        # Strip echoed prompt — keep only the Answer: portion
        answer   = raw.split("Answer:")[-1].strip()

        eval_data["question"].append(q)
        eval_data["answer"].append(answer)
        eval_data["contexts"].append(contexts)
        eval_data["ground_truth"].append(gt)

    ds = Dataset.from_dict(eval_data)
    results = evaluate(
        ds,
        metrics=[faithfulness, context_precision, context_recall],
        llm=ragas_llm,
        embeddings=ragas_embeddings,
    )
    return results, eval_data
```

5.2 Run Evaluation — Section 3.1.3

```
import random
random.seed(42)
sample_indices   = random.sample(range(len(all_questions)), 10)
sample_questions = [all_questions[i] for i in sample_indices]
sample_gts       = [all_answers[i]   for i in sample_indices]

results, eval_data = evaluate_rag_pipeline(sample_questions, sample_gts)

df = results.to_pandas()
print(df[["user_input","faithfulness","context_precision","context_recall"]])
print("\nMeans:", df[["faithfulness","context_precision","context_recall"]].mean())
```

✓ Import changed from ragas.metrics → ragas.metrics.collections to fix deprecation warnings.

6. Free Judge LLM Options

Any of these can replace the local Qwen model as the RAGAS judge. Your RAG chain (answer generation) remains unchanged.

Option 1 — Groq API ★ Recommended

✓ Free tier: 14,400 requests/day · Speed: ~200 tok/s · Setup: one API key

```
# pip install langchain-groq
from langchain_groq import ChatGroq

# Free key at: https://console.groq.com
judge_llm = ChatGroq(
    model="llama-3.1-8b-instant",
    api_key=os.getenv("GROQ_API_KEY"),
    max_tokens=1024,
)
```

Option 2 — Google Gemini API

✓ Free tier available · Speed: fast · Setup: Google AI Studio key

```
# pip install langchain-google-genai
from langchain_google_genai import ChatGoogleGenerativeAI

# Free key at: https://aistudio.google.com
judge_llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.getenv("GOOGLE_API_KEY"),
    max_tokens=1024,
)
```

Option 3 — Ollama (fully local, no API key)

■ Still runs locally — may timeout on very slow GPUs. Use llama3.2 not Qwen.

```
import subprocess, time
subprocess.Popen(["ollama", "serve"])
time.sleep(5)
subprocess.run(["ollama", "pull", "llama3.2"])   # ~2 GB, once

from langchain_ollama import ChatOllama
judge_llm = ChatOllama(model="llama3.2", num_predict=1024, temperature=0)
```

Option 4 — HuggingFace Inference API

■ Slower free tier — higher timeout risk than Groq/Gemini.

```
# pip install langchain-huggingface
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint

endpoint = HuggingFaceEndpoint(
    repo_id="mistralai/Mistral-7B-Instruct-v0.3",
```

```
    huggingfacehub_api_token=HF_TOKEN,   # already in your notebook
    max_new_tokens=1024,
    task="text-generation",
)
judge_llm = ChatHuggingFace(llm=endpoint)
```

Comparison

Option	Speed	Timeout Risk	Setup	Best For
Groq	■ Fastest	Very Low	1 API key	Best overall
Gemini	Fast	Low	1 API key	Google ecosystem
Ollama	Medium	Medium	Pull model	Fully offline
HF Inference API	Slow	High	HF token (have it)	Last resort

7. RAGAS Metrics — What They Measure

Metric	What it measures	Score range	Higher = ?
faithfulness	Are all claims in the generated answer supported by the retrieved context? (Hallucination detection)	0 – 1	Less hallucination
context_precision	Are the retrieved chunks actually relevant to the question? (Retrieval precision)	0 – 1	Better retrieval
context_recall	Does the retrieved context cover all the information needed to answer the question? (Retrieval recall)	0 – 1	More complete retrieval

Expected Score Ranges for This System

• **faithfulness**: 0.3 – 0.6 — Qwen-3B tends to paraphrase and occasionally adds unsupported detail; 128-token cap also truncates answers

• **context_precision**: 0.4 – 0.7 — Legal boilerplate is repetitive so retrieval often finds topically related but not perfectly matching chunks

• **context_recall**: 0.3 – 0.6 — Single-sentence ground truths are often covered; multi-sentence answers may be partially missed with k=3 retrieval

8. Conclusions & Insights

Data Insights

• Legal documents are highly repetitive — boilerplate clauses appear across many contracts, explaining the non-trivial TF-IDF similarity even across different document types.

• Document length varies enormously (142 – 84,946 words). A fixed chunk size of 1,000 characters works but may split critical clauses mid-sentence.

• Top words (company, agreement, section, parent) confirm the dataset is dominated by merger/acquisition contracts from the MAUD subset.

Model & Pipeline Insights

• Qwen2.5-3B is capable of grounded legal Q&A but the 128-token limit frequently cuts answers before the key legal clause is stated.

• all-MiniLM-L6-v2 is efficient but only 384-dimensional; a larger model (e.g., legal-bert or e5-large) would likely improve retrieval precision.

• k=3 retrieval is conservative — increasing to k=5 would improve recall at the cost of slightly noisier context.

Evaluation Insights

• RAGAS requires a capable judge LLM (7B+ parameters or a fast API model). Using the same small local model for both generation and judging causes timeout failures and NaN metrics.

• Separating generation (local Qwen) from judgment (Groq / Gemini) is the correct architecture for cost-effective, reliable RAGAS evaluation.

Recommendations

• Increase max_new_tokens to 256–512 for better answer completeness.

• Use Groq (free) as RAGAS judge LLM to eliminate timeout errors.

• Consider semantic chunking or sentence-aware splitting to avoid breaking legal clauses across chunk boundaries.

• Strip the echoed prompt from LLM output before passing to RAGAS using: answer = raw.split('Answer:')[-1].strip()